CPE 314 - Software Design

University of Cebu

Computer Engineering Department

UC-B Campus Marketplace with Reviews

Project Design Document

Table 1: Group information

Group Name / No.:	Group 1 / Team 01 | BSCPE 3B | EDP 69323
Members:	John Benedict Alaiza
	Acexion B. Mahigugmaon
	Glyde V. Arreglado
	Tristan Dominic Urot
	Joshua Brian S. Enovero
	Eman De Paz
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	V 0.3 / 2 November 2026 - Week 10 design

Table of Contents

Revision History	3
1. Introduction & Project Overview	4
2. Preliminary ER Sketch & Team Roles	4
2.1 Key Attributes and Relationships	5
2.2 Team Roles	5
3. System Architecture & Object-Oriented Design	6
3.1 Runtime behavior and supporting classes	7
4. Data Model & Database Design	8
4.1 Subtype records and cardinalities	9
4.2 Schema description	10
5. API Integration Design	11
6. Concurrency Design	12
7. Data Mining & Visualization Design	13
8. Testing Summary	14
9. Individual Contributions	14
Appendix - Project References	15

Revision History

Table 2: Document revisions

Version	Date	Author	Changes
0.1	23 Oct 2026	Group 1	Initial Week 8 draft: overview, preliminary ER sketch and team responsibilities.
0.2	25 Oct 2026	Group 1	Week 9 revision: simpler classes and GUI, finalized ERD and matching schema.
0.3	2 Nov 2026	Group 1	Week 10 sections expanded: API Integration Design, Concurrency Design, and Data Mining & Visualization Design.

1. Introduction & Project Overview

UC-B Campus Marketplace with Reviews is a local Java desktop prototype for students, faculty and school groups to post and browse campus goods. It follows Option J with Textbook, Electronics and Furniture listings and their different condition rules. Milestone 1 uses JavaFX, SQLite/JDBC, category recursion, buyer-interest waitlists and recently viewed items.

Reviews, FakeStoreAPI suggestions, background JSON sales import, category analytics and charts remain planned for Milestone 2. Campus uniforms and department merchandise remain proposed extensions. The current prototype uses fictional demo members and local storage; authentication and online payments are outside this milestone.

2. Preliminary ER Sketch & Team Roles

The preliminary model carries forward the entities, key attributes and relationships from the proposal. The sketch below shows the core listing relationships; Table 3 lists all seven proposed records and their foreign keys.

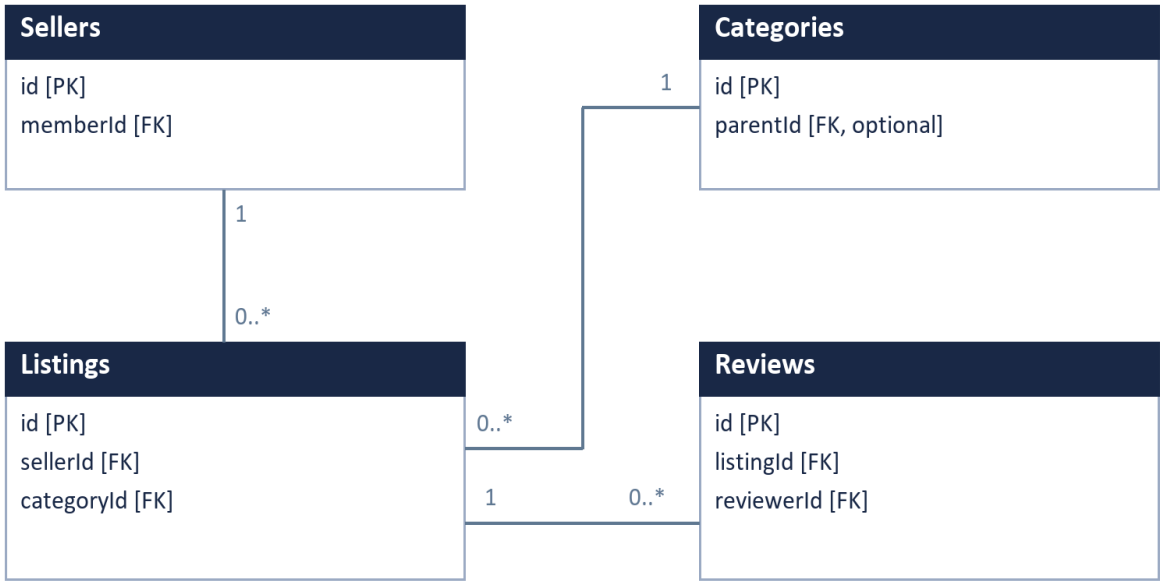


Figure 1: Core preliminary entity relationships

Each seller and category can relate to many listings; each listing can receive many reviews. The 1 and 0..* labels mean one parent and zero or more related records. This is a proposal sketch; the final ERD and actual schema will be documented in Section 4.

2.1 Key Attributes and Relationships

PK identifies a record uniquely. FK refers to a related record. Categories.parentId may be empty for a root category.

Table 3: Preliminary entities and key attributes

Entity	Key attributes
Members	id PK; name
Sellers	id PK; memberId FK → Members.id; displayName; sellerKind
Categories	id PK; parentId FK → Categories.id; name
Listings	id PK; sellerId FK → Sellers.id; categoryId FK → Categories.id; type; title; priceMinor; currency; status
Reviews	id PK; listingId FK → Listings.id; reviewerId FK → Members.id; rating; comment
BuyerInterests	id PK; listingId FK → Listings.id; buyerId FK → Members.id; joiningOrder
HistoricalSales	id PK; categoryId FK → Categories.id; sourceRecordId; priceMinor; currency; soldAt

The Week 8 sketch introduces the seven core records. Listings also have one matching subtype-details record. Section 4 gives the finalized fields, all ten tables and the complete relationships.

2.2 Team Roles

Table 4: Proposed primary responsibilities for Week 8

Member	Primary responsibility
John Benedict Alaiza - Backend Developer (Listing Models)	Sample listings, categories and condition examples.
Acexion B. Mahigugmaon - Project Leader / Integration	Architecture, implementation planning and integration.
Glyde V. Arreglado - Database Admin	ER sketch, attributes and relationship review.
Tristan Dominic Urot - UI/GUI Developer	GUI sketches, control labels and user steps.
Joshua Brian S. Enovero - Backend & API Developer	Sample JSON and API/import validation cases.
Eman De Paz - QA & Testing	Test cases and expected analytics calculations.

These are proposed responsibilities for member confirmation. Actual contributions and supporting commits will be recorded in Section 9.

3. System Architecture & Object-Oriented Design

The working Milestone 1 prototype has 12 Java files. Models hold item rules, a service coordinates actions, a database class handles SQL, and one JavaFX view displays the screen.

Listing (abstract)
- Textbook extends Listing
- Electronics extends Listing
- Furniture extends Listing

Listing stores the shared ID, seller/category IDs, title, price, currency and status. It declares getType() and gradeCondition(). Each subclass stores its condition evidence and implements these methods.

Table 5: Class responsibilities and SRP justification

Class	Single responsibility and reason to change
Listing	Stores and validates common item values and defines the grading contract. Changes when shared item rules change; it contains no SQL or GUI controls.
Textbook	Stores textbook evidence and grades its condition. Changes when textbook condition policy changes.
Electronics	Stores electronics evidence and grades its condition. Changes when electronics condition policy changes.
Furniture	Stores furniture evidence and grades its condition. Changes when furniture condition policy changes.

The table holds a List<Listing>. Calling gradeCondition() through that shared reference runs the actual subclass method. This is runtime polymorphism in the running app.

Implemented condition rules

Textbook: missing pages - POOR; otherwise heavy markings - FAIR; light markings or cover damage - GOOD; otherwise LIKE_NEW.

Electronics: no power or a functional defect - POOR; otherwise cosmetic damage - GOOD; otherwise LIKE_NEW.

Furniture: instability or structural damage - POOR; otherwise heavy wear - FAIR, light wear - GOOD, no wear - LIKE_NEW.

3.1 Runtime behavior and supporting classes

The other eight files support the four listing classes. Their jobs are grouped below.

Table 6: Supporting responsibilities outside the listing hierarchy

Component	Responsibility
App	Starts JavaFX, connects the database, service and screen, and closes the worker on exit.
MarketplaceView	Builds the screen, reads form values, handles button clicks and displays results.
MarketplaceService	Coordinates posting, member ownership, category recursion, waitlists and recent views.
Database	Opens SQLite, creates tables, runs bound SQL and saves/loads listing objects.
Member and Category	Hold member identity and category name/parent information.
ConditionGrade and WearLevel	Define the allowed condition grades and wear values.

Working GUI flow

The single screen has a listing table, inline posting form, category/search controls and details. Post listing saves a typed item and reloads the table with SELECT. View details records a view; Join waitlist adds buyer interest.

Data structures and recursion

Queue: Queue<Long> backed by ArrayDeque displays buyers in FIFO joining order. Saved rows rebuild the queue; offer() adds at the tail and poll() removes the head for display. Saved rows remain unchanged. Duplicate entries are rejected.

Stack: a Map stores one Stack<Long> per member. push() records a view; pop() reads a copy, newest first. History holds at most 20 unique items for the session; three are previewed.

Recursion: collectCategoryIds() calls itself for each child category. A leaf has no children and returns. Electronics includes Computers, Laptops and Gaming Laptops. Missing parents and cycles are rejected before traversal.

Build and background work

Java 21 and Maven provide shared libraries. Saved editor Run profiles launch App directly. One worker runs database operations; JavaFX displays the result on its UI thread.

4. Data Model & Database Design

The implemented SQLite schema follows the Week 8 sketch. Figure 2 shows seven core tables and Figure 3 shows the three subtype tables. Together they match the ten tables defined in schema.sql.

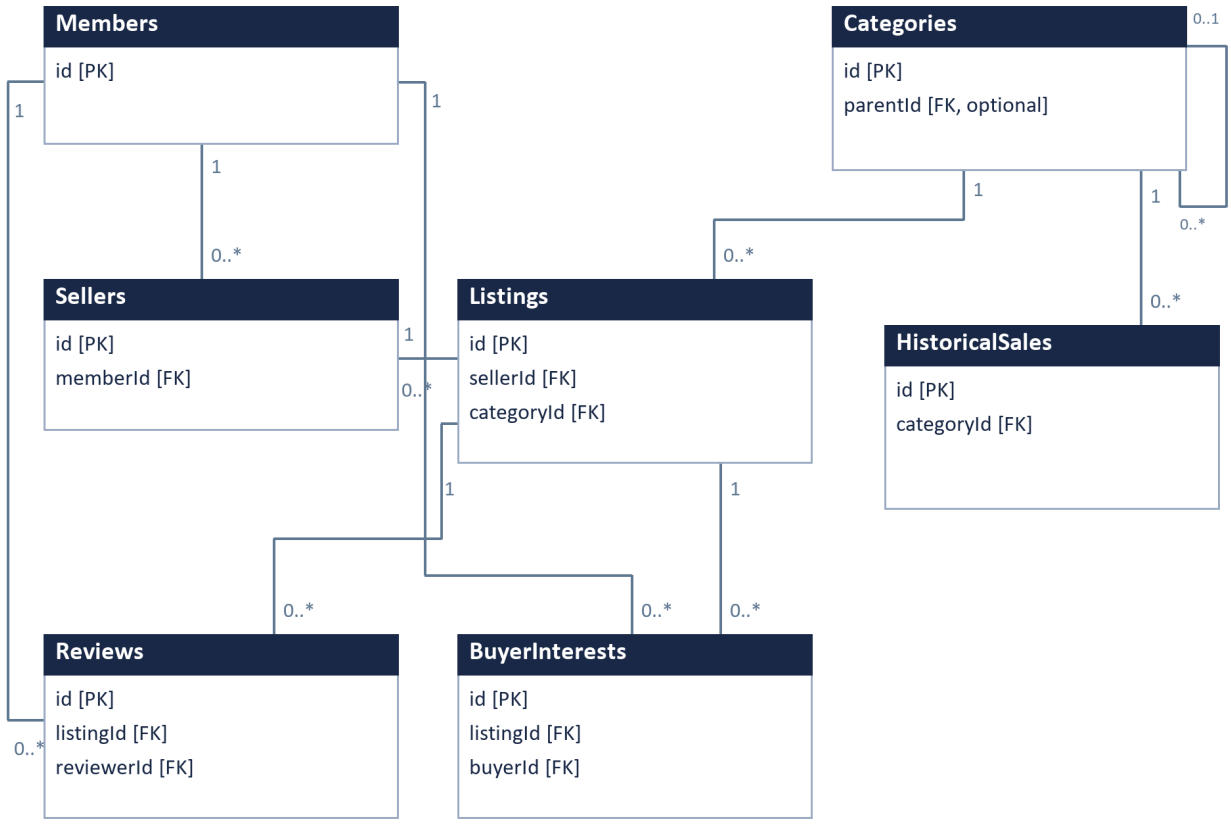


Figure 2: Week 9 core entity relationships

The 1 and 0..* endpoints mean one parent and zero or more child records. A root category has no parent; each non-root category has one parent. A parent can have many children. Every foreign-key relationship is listed explicitly on the following page.

4.1 Subtype records and cardinalities

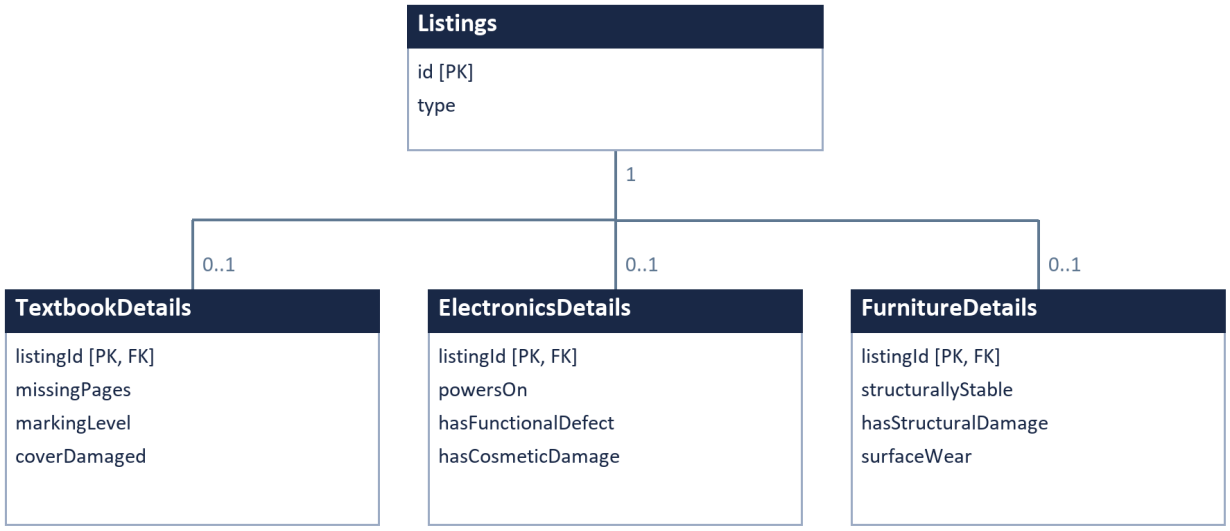


Figure 3: Listing subtype tables with shared primary keys

Each listing has one matching subtype-details row. Database.insertListing() saves both rows in one transaction and rolls back on failure. The load method checks that the detail row matches the listing type. Foreign keys alone do not enforce this rule.

Table 7: Complete relationship cardinalities

Parent	Child	Cardinality
Members	Sellers	1 to 0..*
Members	Reviews	1 to 0..*
Members	BuyerInterests	1 to 0..*
Sellers	Listings	1 to 0..*
Categories	Listings	1 to 0..*
Categories	HistoricalSales	1 to 0..*
Categories	Categories	0..1 parent to 0..* children
Listings	Reviews	1 to 0..*
Listings	BuyerInterests	1 to 0..*
Listings	Each subtype table	1 to 0..1; exactly one matching subtype overall

4.2 Schema description

This table matches src/main/resources/db/schema.sql. IDs are INTEGER; FK means foreign key. Required text values are nonblank. Prices use integer minor units plus a three-letter uppercase currency: PHP 350.50 is stored as 35050 and PHP.

Table 8: SQLite schema matching the implemented ERD

Table	Columns and constraints
Members	id PK; name required TEXT.
Sellers	id PK; memberId required FK to Members; displayName and sellerKind required TEXT.
Categories	id PK; parentId nullable FK to Categories; name required TEXT; parentId cannot equal id.
Listings	id PK; sellerId/categoryId required FKs; type is TEXTBOOK, ELECTRONICS or FURNITURE; title required; priceMinor positive INTEGER; currency three uppercase letters; status nonblank TEXT. Java supports AVAILABLE and WITHDRAWN.
Reviews	id PK; listingId/reviewerId required FKs; rating INTEGER from 1 to 5; comment nullable TEXT. Review entry is Milestone 2.
BuyerInterests	id PK; listingId/buyerId required FKs; joiningOrder positive INTEGER; UNIQUE(listingId, buyerId) and UNIQUE(listingId, joiningOrder).
HistoricalSales	id PK; categoryId required FK; sourceRecordId unique required TEXT; priceMinor positive INTEGER; currency three uppercase letters; soldAt nonblank TEXT. JSON import and timestamp validation are later work.
TextbookDetails	listingId PK/FK to Listings; missingPages nonnegative INTEGER; markingLevel is NONE, LIGHT or HEAVY; coverDamaged is 0/1.
ElectronicsDetails	listingId PK/FK to Listings; powersOn, hasFunctionalDefect and hasCosmeticDamage are each 0/1.
FurnitureDetails	listingId PK/FK to Listings; structurallyStable and hasStructuralDamage are each 0/1; surfaceWear is NONE, LIGHT or HEAVY.

Foreign keys are enabled on every connection. The three detail tables use ON DELETE CASCADE. The retained backend withdrawal method changes status and keeps related history; this smaller GUI has no withdrawal button.

Database.insertListing() commits the listing and its subtype together, or rolls back both. Java validates category cycles. SQL values use PreparedStatement bindings. Later category price calculations will keep currencies separate.

5. API Integration Design

The planned Milestone 2 integration uses **FakeStoreAPI** to help a seller choose a category and compare reference prices. This section describes the design; the API feature is not yet implemented.

API request and JSON response

Java sends **GET https://fakestoreapi.com/products**. This public endpoint needs no API key and returns a JSON array of products. Each product contains id, title, price, description, category and image; it can also include rating.rate and rating.count.

Example response shape, with illustrative values:

```
[
  {
    "id": 1, "title": "Example monitor",
    "price": 120.50, "category": "electronics",
    "description": "Example product details",
    "image": "https://example.com/product.png",
    "rating": {"rate": 4.2, "count": 25}
  }
]
```

Parsing: turning JSON into Java data

A planned ProductCatalogClient uses Java HttpClient to make the request. A JSON parser reads the returned objects and keeps their ID, title, category and price. Prices use BigDecimal. The client checks the HTTP status and required fields before returning results to MarketplaceService.

Meaningful use in the app

The seller enters a title and clicks Suggest category / price. For electronics, the service compares lowercase title words with electronics catalog products and shows up to three best matches. Their lowest/highest prices form a reference range; one match gives one reference price.

The suggested category maps to our local Electronics category. The seller can change it to a child category, enter a final price and post the listing. MarketplaceView displays the suggestions; Database saves the confirmed local listing.

Limits and failure handling

No match, including unsupported textbook/furniture items, keeps manual entry available. A timeout, unsuccessful response or invalid JSON shows a short error and allows retry. The API has no currency field, so label its prices as catalog references with unspecified currency; do not treat them as PHP. PHP entry stays manual.

6. Concurrency Design

The planned long-running operations are **API requests, large JSON sales-history imports and database summaries**. They will run on a background thread so the user can still operate the window.

Threading approach

Use JavaFX **Task** with one **ExecutorService**. A Task holds the slow operation; the executor runs it on a worker thread. The JavaFX application thread reads form values before starting the task and updates controls through success, failure and cancellation callbacks.

The background worker handles HTTP, JSON parsing and substantial SQL work. It does not directly read or change JavaFX controls. Each database operation uses its own connection, with foreign keys enabled.

Background JSON import

The importer reads a UTF-8 JSON array one record at a time. Each record contains a source ID, category, sale price, currency and sale date/time. It checks required values, an existing category, a positive valid price and a valid timestamp; repeated source IDs are skipped.

Prepared SQL inserts save valid records into HistoricalSales. One transaction groups the import: commit saves the new rows after the full file is parsed; rollback removes those new rows if the JSON is broken, the database fails or the user cancels. Existing saved records remain.

How the UI stays usable

Show progress and allow Cancel during import. Disable only actions that conflict with the current job. The loaded listing table can still be scrolled and selected, and form fields can remain editable. Refresh saved data after the task finishes.

The current Milestone 1 helper disables the whole screen during a task. Milestone 2 will replace that with selective disabling. Using another thread alone is not enough if all controls are disabled.

Finishing the operation

The task returns saved, invalid and duplicate counts or a clear error. Show final counts only after commit; after rollback, the saved count is zero. Restore controls on success, failure or cancellation. Closing the app requests cancellation and shuts down the worker.

7. Data Mining & Visualization Design

The planned analysis calculates **average listing price and average review rating by category** from the local database. These are simple summaries of saved data.

Table 9: Category calculations

Result	Data and calculation
Average listing price	Use AVAILABLE Listings. Group by category and currency. Add prices and divide by the number of listings; count each listing once. Convert stored priceMinor to display units by dividing by 100.
Best-rated category	Join campus Reviews to AVAILABLE Listings. Average the individual ratings in each category and show the review count. Highest mean wins; report ties. A category without reviews is unrated.

Which data is included

Use separate queries for price and rating so several reviews do not count a listing price more than once. Group by the listing's own category and keep currencies separate. HistoricalSales contains past sale prices; API prices and API ratings are external catalog data. These do not enter the two listing/review calculations.

Reviews will come from the planned review action: a member rates a listing from 1 to 5. The service rejects self-review and a second review of that listing by the same member.

Charts and why they fit

Use two JavaFX **bar charts** because bars make category values easy to compare:
Average listing price by category: X = Category; Y = Average price in the selected currency, default PHP.
Average review rating by category: X = Category; Y = Mean rating, on a 0-5 axis.

Each chart has a title and axis labels. Show a legend if more than one series is displayed, and a no-data message when results are empty. Chart values come from the queries and are refreshed when data changes.

Small example

PHP 100 and PHP 200 have an average of PHP 150. Adding a PHP 300 listing changes the average to PHP 200. Ratings 4 and 5 average to 4.5. These examples explain the formulas; the app will calculate from its actual stored records.

8. Testing Summary

The previous Milestone 1 checks recorded **27 passing JUnit tests**: 8 model, 10 database and 9 service tests, with zero failures/errors. Existing local reports still show these totals. They are not new Milestone 2 test results.

Table 10: Existing test coverage and edge cases

Test file	Coverage and an edge case
ListingTest.java (8 tests)	Condition grading and common listing values. Invalid prices/values are rejected; subclass-specific grading is checked.
DatabaseTest.java (10 tests)	Table setup, saving/reopening, relationships and transactions. If a subtype insert fails, the parent listing is rolled back.
MarketplaceServiceTest.java (9 tests)	Posting, search, waitlists, history and recursive category search. Cycles and missing category parents are rejected.

Earlier verification also recorded 23 GUI control/flow checks. API, JSON import, review and analytics tests will be added after implementation; their results are not claimed here.

9. Individual Contributions

Section 2 records the assigned team roles. A verified per-member contribution record with commit references has not yet been consolidated for this revision. It will list actual completed changes when available.

Appendix - Project References

Team repository: XionAce/cpe314-69323-capstone-team-01

Approved revised project proposal - Ang I Print

Project Design Document Template

Week 8 Project Design Document Checklist

Week 9 Project Design Document Checklist

Week 10 Project Design Document Checklist

FakeStoreAPI documentation: https://fakestoreapi.com/docs

API maintainer reference: https://github.com/keikaavousi/fake-store-api